

Constructor, Operator "new" - Simple Hinglish Summary

Source: <https://javascript.info/constructor-new>

Main baat

Normal `{...}` syntax se **ek object** banta hai. Lekin aksar hume **kai milte-julte objects** chahiye hote hain (kai users, menu items, etc.). Iske liye **constructor functions** aur `new` **operator** use hota hai.

1. Constructor function

Constructor functions technically **normal functions** hi hain. Bas do conventions hain:

1. Unka naam **capital letter** se shuru hota hai.
2. Unhe sirf `new` **operator ke saath** chalana chahiye.

javascript

```
function User(name) {  
  this.name = name;  
  this.isAdmin = false;  
}  
  
let user = new User("Jack");  
  
alert(user.name);    // Jack  
alert(user.isAdmin); // false
```

`new` **ke saath function chalne par kya hota hai?**

1. Ek **naya khali object** banta hai aur `this` ko assign hota hai.
2. Function ki body chalti hai. Aam taur par wo `this` me nayi properties jodti hai.
3. `this` ki value return hoti hai.

Yaani `new User(...)` kuch aisa karta hai:

javascript

```
function User(name) {  
  // this = {}; (implicitly)  
  
  // this me properties jodo  
  this.name = name;  
  this.isAdmin = false;  
  
  // return this; (implicitly)  
}
```

To `(let user = new User("Jack"))` ka result ye hai:

javascript

```
let user = {  
  name: "Jack",  
  isAdmin: false  
};
```

Ab dusre users bhi `(new User("Ann"))`, `(new User("Alice"))` se ban sakte hain. Har baar literal likhne se **bahut chhota aur readable**.

Constructors ka main maqsad: reusable object creation code.

Note: Technically **koi bhi function** (arrow functions ko chhodkar, kyunki unka `(this)` nahi hota) constructor ki tarah use ho sakta hai. "Capital letter" bas ek common agreement hai jisse pata chale ki ye function `(new)` ke saath chalana hai.

```
new function() { ... }
```

Ek hi complex object banane ka bahut sara code ho to use ek turant-call hone wale constructor me wrap kar sakte hain:

javascript

```
// function banao aur turant new se call karo  
let user = new function() {  
  this.name = "John";  
  this.isAdmin = false;  
  
  // ...user banane ka baaki code  
  // complex logic, statements  
  // local variables etc  
};
```

Ye constructor dobara call nahi ho sakta kyunki wo kahin save nahi hai. Ye trick sirf **ek object ko banane wale code ko encapsulate** karne ke liye hai, reuse ke liye nahi.

2. Constructor mode test: `new.target` (advanced)

┆ Ye syntax bahut kam use hota hai. Poori jaankari chahiye tabhi padho.

Function ke andar `new.target` se check kar sakte hain ki wo `new` ke saath call hua ya bina `new` ke:

- Normal call me `undefined`
- `new` ke saath call hone par function khud

javascript

```
function User() {  
  alert(new.target);  
}  
  
// bina "new" ke:  
User(); // undefined  
  
// "new" ke saath:  
new User(); // function User { ... }
```

Isse `new` aur normal call dono ko same kaam karne wala bana sakte hain:

javascript

```
function User(name) {  
  if (!new.target) { // agar bina new ke chalaya  
    return new User(name); // ...to main new laga dunga  
  }  
  
  this.name = name;  
}  
  
let john = User("John"); // new User par redirect ho gaya  
alert(john.name); // John
```

Libraries me kabhi kabhi aisa hota hai. Lekin **har jagah use karna achha nahi**, kyunki `new` na likhne se ye kam obvious hota hai ki naya object ban raha hai.

3. Constructors se return

Aam taur par constructors me `return` nahi hota. Unka kaam `this` me sab kuch likhna hai, aur wo automatically result ban jata hai.

Lekin `return` ho to rule simple hai:

- `return` ke saath object diya to wo object return hota hai, `this` ki jagah.
- `return` ke saath primitive diya to wo ignore ho jata hai.

Yaani object ke saath `return` wo object deta hai, baaki sab cases me `this` return hota hai.

Object return karne se `this` override ho jata hai:

```
javascript

function BigUser() {

    this.name = "John";

    return { name: "Godzilla" }; // <-- ye object return hoga
}

alert( new BigUser().name ); // Godzilla
```

Khali `return` (ya primitive) ho to `this` return hota hai:

```
javascript

function SmallUser() {

    this.name = "John";

    return; // <-- this return hoga
}

alert( new SmallUser().name ); // John
```

Aam taur par constructors me `return` nahi hota. Ye special behavior sirf पूरी जानकारी के liye hai.

`new` के बाद brackets hata sakte hain

```
javascript
```

```
let user = new User; // <-- bina brackets ke
// same as
let user = new User();
```

Brackets hatana **achhi style nahi** maana jata, lekin specification isse allow karti hai.

4. Constructor me methods

Constructor functions se objects banane me bahut flexibility milti hai. Constructor ke **parameters** tay kar sakte hain ki object kaise bane. `this` me sirf properties hi nahi, **methods** bhi jod sakte hain.

javascript

```
function User(name) {
  this.name = name;

  this.sayHi = function() {
    alert( "My name is: " + this.name );
  };
}

let john = new User("John");

john.sayHi(); // My name is: John

/*
john = {
  name: "John",
  sayHi: function() { ... }
}
*/
```

Complex objects banane ke liye aur advanced syntax **classes** hai, jo aage aayegi.

Summary

- **Constructor functions** normal functions hain, bas naam **capital letter** se shuru hone ka agreement hai.
- Constructor functions ko **sirf** `new` se call karna chahiye. Aisa call shuru me khali `this` banata hai aur end me bhara hua `this` return karta hai.

Constructor functions se **kai milte-julte objects** bana sakte hain.

JS me kai **built-in objects** ke liye bhi constructors hain: `Date` (dates), `Set` (sets), aur bhi.

Objects par hum wapas aayenge: is chapter me sirf basics hain. Aage "Prototypes, inheritance" aur "Classes" chapters me detail me padhenge.

Practice Tasks (Answers)

1. Do functions, ek object: kya aisi **A** aur **B** functions bana sakte hain jinke liye `new A() == new B()`?

Haan, possible hai. Function agar object return kare to `new` `this` ki jagah wahi object return karta hai. Dono same bahar ke object ko return kar sakte hain:

javascript

```
let obj = {};  
  
function A() { return obj; }  
function B() { return obj; }  
  
alert( new A() == new B() ); // true
```

2. Naya Calculator (constructor se):

javascript

```
function Calculator() {  
  
    this.read = function() {  
        this.a = +prompt('a?', 0);  
        this.b = +prompt('b?', 0);  
    };  
  
    this.sum = function() {  
        return this.a + this.b;  
    };  
  
    this.mul = function() {  
        return this.a * this.b;  
    };  
}  
  
let calculator = new Calculator();  
calculator.read();  
  
alert( "Sum=" + calculator.sum() );  
alert( "Mul=" + calculator.mul() );
```

3. Naya Accumulator:

javascript

```
function Accumulator(startingValue) {
  this.value = startingValue;

  this.read = function() {
    this.value += +prompt('How much to add?', 0);
  };
}

let accumulator = new Accumulator(1);
accumulator.read();
accumulator.read();
alert(accumulator.value);
```

Quick Summary

Baat	Yaad rakho
Constructor	Capital letter se shuru hone wala function, <code>new</code> se chalta hai
<code>new</code> kya karta hai	Khali <code>this</code> banata hai → body chalti hai → <code>this</code> return karta hai
Purpose	Reusable object creation code
Arrow function	Constructor nahi ban sakta (<code>this</code> nahi hota)
<code>return object</code>	Wo object return hota hai (<code>this</code> nahi)
<code>return primitive</code>	Ignore hota hai, <code>this</code> return hota hai
<code>new.target</code>	<code>new</code> ke saath call hua ya nahi (advanced)
<code>new User</code> (bina brackets)	Chalta hai, lekin achhi style nahi
Methods	<code>this.sayHi = function() {...}</code>